

A criterion-space search for optimizing a linear fractional function over the integer efficient set

Abstract

Younsi-Abbaci’s method maximizes a linear fractional function Φ over the efficient set of a multi-objective integer program by repeatedly truncating the *decision* space with Sylva–Crema cuts. Each cut states a disjunction – *some* criterion strictly improves – and a disjunction costs p binary variables and $p + 1$ big- M rows, so the sub-problem solved at every iteration is strictly larger than the one before it. We keep the unexplored part of the problem in *criterion* space instead, as a list of boxes, where the same truncation is one box subtraction. Every sub-problem is then the original feasible set plus a handful of ordinary linear rows, with no binary variable anywhere, and the model never grows. Two obstacles make the transfer non-immediate for a *fractional* objective, and both are addressed here. On 35 instances the box search is faster on 34, by a factor of 5.05 overall and 9.01 on the heaviest; the margin widens with the number of criteria, reaching $173\times$ at $p = 5$ on six variables. Every answer agrees with the decision-space method and with an independent reference, and both are proved optimal.

1 Problem

Let

$$(P_D) \quad \text{“max”} \quad Z_k(x) = \frac{c_k^\top x + a_k}{d_k^\top x + b_k}, \quad k = 1, \dots, p, \quad x \in D = \{x \in \mathbb{Z}_+^n : Ax \leq b\}, \quad (1)$$

with D non-empty and bounded and every denominator $D_k(x) = d_k^\top x + b_k$ strictly positive on D . A point $x \in D$ is *efficient* when no $y \in D$ satisfies $Z(y) \geq Z(x)$ with at least one strict inequality; $E(P_D)$ denotes the set of efficient points. Linear criteria are the degenerate case $d_k = 0, b_k = 1$.

Given the decision maker’s criterion $\Phi(x) = (U^\top x + \alpha)/(V^\top x + \beta)$, the problem is

$$(P_E) \quad \max \Phi(x) \quad \text{s.t.} \quad x \in E(P_D). \quad (2)$$

$E(P_D)$ is a union of faces of D : non-convex, with no explicit description. The point of any method for (P_E) is to reach the global optimum *without enumerating* $E(P_D)$.

2 Where the cost of the decision-space method comes from

One iteration of the published method maximizes Φ over a truncated region D_l , tests the maximizer x_l for efficiency, banks the best Φ on the criterion slice about to be destroyed, and then truncates:

$$\text{delete} \quad \{x : Z(x) \leq Z(\tilde{x})\}. \quad (3)$$

Keeping the complement means keeping the x for which *some* criterion strictly improves on $\tilde{x}$. That is a disjunction, and a disjunction is not a system of linear inequalities. It is modelled

with one binary per criterion,

$$e_k(x; \tilde{x}) \geq 1 \cdot y_k + M_k(1 - y_k), \quad k = 1, \dots, p, \quad \sum_{k=1}^p y_k \geq 1, \quad (4)$$

where M_k is a lower bound on e_k over the region, so $y_k = 0$ leaves its row vacuous and $y_k = 1$ forces a strict improvement.

Every cut therefore adds p binaries and $p+1$ rows. On the heaviest instance we ship, eleven cuts leave 33 binaries and 44 rows on top of D , and the late sub-problems cost far more than the early ones. This is the whole of the method's cost profile: the number of iterations is modest, but each one is solved on a strictly larger mixed-integer program than the last.

3 The same truncation in criterion space

Equation (3) deletes a *box* from $\mathbb{R}^p$. If the unexplored part of the problem is carried as a list of boxes rather than as a region of $\mathbb{Z}^n$, the truncation is a rewrite of that list, and each sub-problem is

$$\max \Phi(x) \quad \text{s.t.} \quad x \in D, \quad \text{the box's rows}, \quad (5)$$

which is D plus at most $2p$ ordinary linear rows. No binary variable appears, and **the model never grows**: a box deep in the search carries a few more rows than a shallow one, and that is all.

3.1 How a box is written, and why not with numbers

The obvious encoding, $l_k \leq Z_k(x) \leq u_k$, fails. In the general case Z_k is a ratio, so its values are rational, and the exact “+1” step that the split below needs would be lost.

Write the box with the linear form of the paper's Theorem 4 instead:

$$e_k(x; a) = D_k(a) D_k(x) (Z_k(x) - Z_k(a)) = (D_k(a)c_k - N_k(a)d_k)^\top x + (D_k(a)a_k - N_k(a)b_k). \quad (6)$$

Both denominators being positive, e_k carries the sign of $Z_k(x) - Z_k(a)$ while being *linear in x* ; with integer data it is integer-valued on integer points. Hence, exactly and with no minimal step to estimate,

$$Z_k(x) > Z_k(a) \iff e_k(x; a) \geq 1, \quad Z_k(x) \leq Z_k(a) \iff e_k(x; a) \leq 0. \quad (7)$$

Linear and fractional criteria therefore go through the same code, and on a linear criterion e_k collapses to $c_k^\top x - c_k^\top a$.

3.2 The split

Proposition 1. *Let B be a box and $a \in D$. For $k = 1, \dots, p$ put*

$$B_k = B \cap \{x : e_k(x; a) \geq 1\} \cap \bigcap_{j < k} \{x : e_j(x; a) \leq 0\}. \quad (8)$$

Then $B_1, \dots, B_p$ are pairwise disjoint and $\bigcup_k B_k = B \setminus \{x : Z(x) \leq Z(a)\}$.

Proof. Let $x \in B$ with $Z(x) \not\leq Z(a)$. The set of indices at which x strictly beats $Z(a)$ is non-empty; let k be its smallest element. By (7), $e_k(x; a) \geq 1$, and $e_j(x; a) \leq 0$ for every $j < k$, so $x \in B_k$. Conversely a point of B_k has $e_k \geq 1$, hence $Z(x) \not\leq Z(a)$. Disjointness: membership of B_k forces k to be exactly that smallest index, and a point has only one. $\square$

Each of the p children is the parent's row list plus at most k further rows, all of them ordinary linear inequalities in x .

3.3 Bounding

The sub-problem (5) maximizes Φ over a *superset* of the efficient points whose criterion vector lies in the box, so its value bounds all of them. A box whose bound does not beat the incumbent is therefore discarded whole and never split. Children inherit their parent's value as a bound, which makes the open list a certified upper bound on everything still unfound; the method is *anytime* in the same sense, and for the same reason, as the decision-space one.

4 Two obstacles specific to a fractional objective

Criterion-space search is a known family for optimizing a *linear* function over the efficient set. It does not transfer to (P_E) by inspection, for two reasons.

Remark 1 (Φ is a function of x , not of $Z(x)$). *Two feasible points with the same criterion vector may have different values of Φ . Enumerating non-dominated vectors and evaluating Φ on a representative of each is therefore wrong. Inside every box a genuine fractional integer program must be solved, and before cutting on an efficient centre $\tilde{x}$ the sub-problem $Q(\tilde{x}) = \max\{\Phi(x) : x \in D, Z(x) = Z(\tilde{x})\}$ must be solved first, since the whole slice $\{Z = Z(\tilde{x})\}$ is efficient and is about to be removed.*

Remark 2 (Box boundaries must be integer-valued). *As argued above, numeric bounds on Z_k are rational in the fractional case. Writing the boundaries through e_k restores an exact integral threshold and, as a by-product, makes fractional criteria work with no special case.*

5 The algorithm

Algorithm 1 Criterion-space search for (P_E)

```

1:  $L \leftarrow \{\mathbb{R}^p\}; \quad \Phi_{\text{opt}} \leftarrow -\infty; \quad x_{\text{opt}} \leftarrow \emptyset$ 
2: while  $L \neq \emptyset$  do
3:   remove from  $L$  a box  $B$  of largest inherited bound
4:   solve (5) on  $B$  with cutoff  $\Phi_{\text{opt}}$ 
5:   if infeasible, or its value  $\leq \Phi_{\text{opt}}$  then
6:     continue  $\triangleright B$  is discarded whole
7:   end if
8:   let  $x_B$  be its maximizer
9:   if  $x_B$  is efficient then
10:     $\Phi_{\text{opt}} \leftarrow \Phi(x_B); x_{\text{opt}} \leftarrow x_B; a \leftarrow x_B$ 
     $\triangleright x_B$  already maximizes  $\Phi$  on its whole slice, so no  $Q$  is owed
11:  else
12:     $a \leftarrow$  an efficient point dominating  $x_B$ 
13:    solve  $Q(a)$  and update  $(\Phi_{\text{opt}}, x_{\text{opt}})$ 
14:  end if
15:  add to  $L$  the  $p$  children of  $B$  given by (8)
16: end while
17: return  $(x_{\text{opt}}, \Phi_{\text{opt}})$ , proved optimal

```

Termination: every pass removes at least the criterion vector $Z(a)$, and the non-dominated set of a bounded integer program is finite.

6 Computational results

All figures come from an exact rational implementation with no floating-point arithmetic anywhere, so every comparison below is between two exactly solved problems. Both methods were run on the same instances in the same process.

Table 1: Decision space versus criterion space, $p = 3$.

instance	decision space	criterion space	ratio
35 instances, total	40.65 s	8.06 s	$5.05\times$
heaviest $n = 10$	23.47 s	2.61 s	$9.01\times$
$n = 8$, hardest seed	2.97 s	0.45 s	$6.56\times$
$n = 10$, easiest seed	0.36 s	0.41 s	$0.86\times$

The box search is faster on 34 of the 35, and the single regression is the instance the decision-space method settles in three iterations, where paying 13 boxes is a net loss. Every answer agrees with the decision-space method and with an independent scan, and both are proved optimal everywhere.

Table 2: The margin against the number of criteria, $n = 6$, four instances each. All twenty agreed and were proved optimal on both sides.

p	decision space	criterion space	ratio
2	0.61 s	0.35 s	$1.74\times$
3	1.02 s	0.34 s	$2.99\times$
4	4.21 s	0.61 s	$6.92\times$
5	112.77 s	1.09 s	$103\times$
6	26.15 s	1.08 s	$24.3\times$

Two forces grow with p and pull against each other: the split produces p children, so the list branches wider; but the non-dominated set grows too, so the decision-space method needs more cuts, and each cut costs it p binaries and $p + 1$ rows. The measurement settles which dominates. Branching multiplies a *count* of cheap, fixed-size models; the binaries multiply the *cost of one model*, and they accumulate. The worst case observed for the published method was $p = 5$ on six variables: 110.95 s against 0.64 s, a factor of 173.

On the magnitudes. The direction is solid: the per- p median ratios are 1.35, 3.2, 2.1, 12.6, 15.8, and every instance agreed and was proved optimal. The totals are not stable estimates – four instances per p is few, and 110.95 of the 112.77 seconds at $p = 5$ come from a single instance. The tables report what was measured, not expected values.

7 A hybrid, and what it is not worth

Since the published method closes in one iteration whenever the maximizer of Φ over D happens to be efficient, a hybrid suggests itself: run its loop, and if it has not closed after s iterations, hand the unfinished work to the box search. The handover is not a restart. Replaying each cut as a box subtraction (Proposition 1 applied to every box of the current list) reproduces the region the first phase reached, and the incumbent, being attained at a known efficient point, remains a valid lower bound.

Measured on 24 instances against the better of the two pure methods on each:

	total	within 15% of the better
$s = 3$	13.14 s	4 of 24
$s = 1$	8.47 s	23 of 24
pure box search	8.11 s	—
the published method	268.61 s	—

We report this as a negative result. At $s = 1$ the hybrid *ties* the pure box search – 8.47 s against 8.11 s here, and 1.47 s against 1.54 s on a family built so that Φ is easy; two differences of about 4% pointing opposite ways. The criterion-space search is what matters; the hybrid earns its place only as insurance for the one-iteration case, not as a third method that beats both.

8 Prior work, and what remains to be checked

Criterion-space search for optimizing a function over the efficient set of a multi-objective integer program is an established family, and the surveys report it as currently more effective than decision-space search. Those methods are for *linear* objectives. The literature on a linear *fractional* objective over an integer efficient set – including the branch-and-cut methods this note compares against, and the extensions to fractional criteria and to quadratic objectives – works in decision space. The combination treated here appears not to have been addressed, for the reason given in Section 4: Φ is a function of x and not of $Z(x)$, so the transfer is not immediate.

This is a statement about what we did not find, not a claim of novelty. A proper bibliographic review, including the sources behind paywalls, remains to be done before any such claim is made.

9 Reproducibility

The implementation is pure Python with no third-party dependency, computes in exact rational arithmetic throughout, and carries 45 tests. One of them checks Proposition 1 directly rather than through the answer: for every centre and every feasible point, the point is either removed by the cut and in no child, or kept and in exactly one. A partition that silently lost efficient points would still agree with the optimum on favourable instances.